



Pull request review —  
non-collateral markets

# Auditor — Non-Collateral Markets

September 17, 2026

# Contents

|                              |           |
|------------------------------|-----------|
| <b>1. Document Revisions</b> | <b>3</b>  |
| <b>2. Introduction</b>       | <b>3</b>  |
| <b>3. System Overview</b>    | <b>4</b>  |
| <b>4. Methodology</b>        | <b>6</b>  |
| <b>5. Findings</b>           | <b>8</b>  |
| 5.1. Major.....              | 8         |
| 5.2. Moderate .....          | 8         |
| 5.3. Minor.....              | 9         |
| <b>6. Conclusion</b>         | <b>26</b> |
| <b>About Us</b>              | <b>28</b> |
| <b>Contacts</b>              | <b>28</b> |

## 1. Document Revisions

| #   | Date               | Author              | Description                                                                                                                                                        |
|-----|--------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1 | September 8, 2026  | Dmitry Khovratovich | Initial report draft.                                                                                                                                              |
| 1.0 | September 17, 2026 | Dmitry Khovratovich | Public report. Client responses incorporated. Two findings withdrawn after independent verification of the client's objections. Findings renumbered consecutively. |

## 2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

ABDK Consulting was engaged by Exactly to review a single change to the protocol's Auditor contract: support for markets whose deposits do not count as collateral. The review covered pull request exactly/protocol #815, "auditor: support non-collateral markets", at commit `6c022f3b046ccbef0b35e2aa3cd32955e66ddf91` of branch `non-collateral-markets`, measured against base commit `4b5fec78f02825d34b15276292fef804607e375e` of `main`. The client declared `contracts/Auditor.sol` as the scope, and the reviewed artifact is the full-context difference between the two commits of that file.

Because the engagement is scoped to a pull request rather than to a whole codebase, findings were assessed against the change itself. Observations whose subject matter lies entirely in lines the pull request did not touch were recorded for the client but are not carried into this report, since they concern pre-existing code that was not commissioned for review. Several such observations were raised and delivered separately.

Files other than `contracts/Auditor.sol` were read only as context, and are not part of the audited scope. These include `Market.sol`, `MarketExtension.sol`, `utils/IPriceFeed.sol`, and the verified/ subclasses `VerifiedAuditor.sol` and `VerifiedMarket.sol`. Reading them was necessary to establish what quantity the new flag excludes and to confirm the behaviour of the paths that consume it, but no assurance is offered on their contents. The pull request also modifies six periphery contracts, deployment configuration, and the test suite; none of that was in scope.

A draft of this report was delivered to the client, who responded to every finding. The client's response is reproduced verbatim beneath each finding. Where a fix was claimed, the referenced commit was examined. Where the client disputed a finding, the technical substance of the objection was independently verified against the source before the finding's disposition was settled; two findings were withdrawn on that basis, and the reasoning is summarised in the conclusion. This review is a static reading of the source. No code was compiled, executed, or tested as part of it.

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

### 3. System Overview

The Auditor contract is the Exactly protocol's risk engine. It tracks, via a per-account bitmap in `accountMarkets`, which Market contracts an account has entered; computes account liquidity and health in `accountLiquidity` from each market's balances, adjust factors, and Chainlink-style price feeds obtained through `IPriceFeed`; and gates borrows, withdrawals, and transfers through shortfall checks in `checkShortfall`. It also bounds how much a liquidator may seize through `checkLiquidation` and clears accounts whose remaining collateral cannot cover outstanding debt through `handleBadDebt`. Per-market configuration, including the price feed and adjust factor, is held in the `MarketData` struct inside the `markets` mapping, with listed markets additionally recorded in `marketList`.

Under review is pull request `exactly/protocol #815`, "auditor: support non-collateral markets", at commit `6c022f3b046ccbef0b35e2aa3cd32955e66ddf91` of branch `non-collateral-markets`, taken against base commit `4b5fec78f02825d34b15276292fef804607e375e` of `main`. The change lets a market be listed so that deposits in it no longer count toward borrowing capacity. Such deposits continue to earn yield and can still back borrowing from other markets, remain freely withdrawable regardless of the account's health, and are excluded from what a liquidator may seize; borrowing from the market itself is unaffected

and any resulting debt is treated like debt in any other market. The change is confined to `contracts/Auditor.sol` and is carried by a new `bool nonCollateral` field added to `MarketData`, alongside a new `nonCollateral` argument on `enableMarket` used to set the flag at listing time, a new admin-only `setNonCollateral` function to toggle it afterwards, and a new `NonCollateralSet` event emitted from both paths whenever the flag is set.

The four guards described by the client are each implemented as a conditional on `markets[market].nonCollateral` (read as `m.nonCollateral` or `marketData.nonCollateral` depending on the function). In `accountLiquidity`, a market's balance is added to `sumCollateral` only when it is not flagged non-collateral, and a simulated withdrawal from such a market is likewise excluded from the hypothetical debt added to `sumDebtPlusEffects`. In `checkShortfall`, the existing early return for accounts that have not entered the market is extended so that the hypothetical liquidity check is bypassed outright whenever the target market is flagged non-collateral, making withdrawals and transfers from it unconditional. In `checkLiquidation`, the accumulation of `totalCollateral`, `adjustedCollateral`, and the per-market `seizeAvailable` amount is wrapped in a check that the market is not non-collateral, while the corresponding debt accumulation is left unconditional, so a non-collateral market's balance can never be seized in a liquidation even though its debt still counts.

In `handleBadDebt`, the per-market check that aborts clearing an account's debt while it still holds redeemable collateral value, computed via `market.maxWithdraw`, is likewise wrapped so that it is only evaluated for markets not flagged non-collateral. Deposits in a non-collateral market therefore do not prevent an account's bad debt from being written off. Together these four changes make a non-collateral market's supply behave as a yield-bearing, always-withdrawable, non-seizable position that plays no part in collateral accounting while its borrowing and debt mechanics remain those of an ordinary market.

## / (root directory)

`Auditor.sol.diff`

`Market.sol`

`MarketExtension.sol`

The client stated four properties that a non-collateral market's supply should have: it earns yield but adds nothing to borrowing capacity; it is always freely withdrawable regardless of the account's health; it can never be seized in a liquidation; and borrowing from the market still works normally against collateral held elsewhere, with the resulting debt counted like any other debt. Each was checked against the implementation and each holds.

Two of them are worth stating precisely, because the mechanism is indirect. Non-seizability is not enforced at the point where a seize is authorised: `checkSeize` tests only that both markets are listed. It follows instead from `checkLiquidation` never populating `seizeAvailable` for a flagged market, which drives `maxRepayAmount` to zero, which `Market.liquidate` then rejects with `ZeroRepay`. The `VerifiedAuditor` subclass overrides the two functions this chain depends on, and its overrides preserve the outcome. Free withdrawal likewise follows from an early return in `checkShortfall`, which `Market.withdraw` and `Market.redeem` reach directly and `transfer` and `transferFrom` reach through `MarketExtension`.

It is also worth recording what the change does not introduce, since this materially shaped the findings. The three functions that decide whether a balance counts as

collateral, whether it may be seized, and whether bad debt may be cleared — `accountLiquidity`, `checkLiquidation`, and `handleBadDebt` — each iterate only the markets recorded in the account's `accountMarkets` bitmap. A deposit in a market the account has never entered is therefore already invisible to all three, and `checkShortfall` already returns early for it. The four properties above consequently already hold, today, for any un-entered market. What the flag adds is not a new class of balance but the ability to place an already-entered position into that class by administrative action, which is a transition no sequence of user operations can produce.

That transition is the one behaviour of the change that has no pre-existing analogue, and it is governed by `setNonCollateral`, which is restricted to `DEFAULT_ADMIN_ROLE` and, per the client, executes only through a public proposal, a vote, and a timelock. Accounts that had borrowed against the market lose their collateral credit at the moment the flag is set, while retaining the ability to withdraw the balance in full. One finding in this report concerned the accuracy of the documentation that warns operators about this. The client has since corrected that documentation, and has committed to assessing outstanding collateral-backed exposure before proposing the flag on any live market.

## 4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the Auditor — Non-Collateral Markets project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

### General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

### Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

**Access Control Analysis**

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

**Code Logic Analysis**

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

**Critical**

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

**Major**

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

**Moderate**

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

**Minor**

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

## 5. Findings

We have completed our technical audit of the Auditor — Non-Collateral Markets code-base and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

|              | Found     | Fixed    |
|--------------|-----------|----------|
| <b>Minor</b> | <b>12</b> | <b>4</b> |

Fixed 4 out of 12 issues reported.

### 5.1. Major

### 5.2. Moderate



## 5.3. Minor

### CVF-1: acknowledged

Category: **flaw**

`adjustFactor` is never validated against zero

`adjustFactor` acts as a per-asset volatility haircut that is applied symmetrically: collateral is discounted by multiplying with it, and debt is surcharged by dividing by it. A value of zero corresponds to infinite volatility, which both zeroes the collateral contribution and makes the debt surcharge undefined. Concretely, the debt-side `divWadUp(adjustFactor)` in `accountLiquidity` and `checkLiquidation` reverts when `adjustFactor == 0`, because `solmate's mulDivUp` reverts on a zero denominator. Since both functions iterate over every market in the account's bitmap, a single market listed with `adjustFactor == 0` would make any account holding that market's bit unable to borrow, withdraw, transfer, be liquidated, or have bad debt cleared.

Neither `enableMarket` nor `setAdjustFactor` rejects a zero (or otherwise out-of-range) value, so this depends entirely on the admin supplying a sane parameter.

This is a pre-existing robustness gap rather than something introduced by the new non-collateral functionality. The debt-side computation is unchanged by the diff, and there is no natural reason to pair `nonCollateral: true` with `adjustFactor == 0`: a non-collateral market is still borrowable, so its debt still needs a finite volatility surcharge, and setting the factor to zero would forbid the asset as debt altogether. The `nonCollateral` flag already expresses that the asset grants no collateral power, so the factor should keep a normal risk-based value there.

A secondary, minor point: the `@param adjustFactor` text on `enableMarket` and the `MarketData.adjustFactor` doc describe the factor only in terms of "lending power" (the collateral side). For a non-collateral market the factor no longer affects collateral but still surcharges debt, which is consistent with the volatility interpretation but not obvious from the collateral-centric wording.

### Recommendation

Add a defensive bounds check where `adjustFactor` is written, in both `enableMarket` and `setAdjustFactor`, so an out-of-range value cannot be stored:

```
error InvalidAdjustFactor(Market market, uint128 adjustFactor);

if (adjustFactor == 0 || adjustFactor > 1e18) revert
    InvalidAdjustFactor(market, adjustFactor);
```

The upper bound guards against a factor above  $1e18$ , which would inflate collateral and deflate debt.

Optionally, broaden the `adjustFactor` documentation on `enableMarket` and `MarketData` to note that the factor discounts collateral and surcharges debt, and that for a non-collateral market it still applies to the debt side.

### Client Comment

Documentation broadened at 39caa02e3fa1bf45cdaae8e9c8a5957210604660.

The missing bounds check is pre-existing behaviour outside the scope of this change and is left as is.

### /Auditor.sol.diff

```
134→134 |         // sum all the debt
135→135 |         sumDebtPlusEffects += vars.borrowBalance.mulDivUp(vars.price,
baseUnit).divWadUp(adjustFactor);
```

same file

```
141 +         if (withdrawAmount != 0 && !m.nonCollateral) {
142→142 |             sumDebtPlusEffects +=
withdrawAmount.mulDivDown(vars.price,
baseUnit).mulWadDown(adjustFactor);
```

same file

```
223→223 |         uint256 value = debt.mulDivUp(m.price, m.baseUnit);
224→224 |         base.totalDebt += value;
225→225 |         base.adjustedDebt += value.divWadUp(m.adjustFactor);
```

same file

```
370→374 |     /// @param adjustFactor market's adjust factor for the underlying
asset.
375 +     /// @param nonCollateral true to list the market as ineligible as
collateral.
```

same file

```
379 +     uint128 adjustFactor,
```

same file

```
399→407 |     function setAdjustFactor(Market market, uint128 adjustFactor)
public onlyRole(DEFAULT_ADMIN_ROLE) {
400→408 |         if (!markets[market].isListed) revert MarketNotListed();
401→409 |
402→410 |         markets[market].adjustFactor = adjustFactor;
403→411 |         emit AdjustFactorSet(market, adjustFactor);
```

same file

```
455→479 |     /// @param adjustFactor used to asses the lending power of the
market's underlying asset.
```

**CVF-2: fixed**Category: **behavior**

NonCollateralSet is emitted conditionally on listing and unconditionally on no-op updates

The new event is emitted inconsistently at the two sites that write the flag.

In enableMarket it is guarded, unlike its three neighbours:

```
emit MarketListed(market, decimals);
emit PriceFeedSet(market, priceFeed);
emit AdjustFactorSet(market, adjustFactor);
if (nonCollateral) emit NonCollateralSet(market, true);
```

Because the surrounding events are emitted unconditionally, a consumer reconstructing market configuration purely from the log sees an explicit initial value for the price feed and the adjust factor but nothing at all for collateral eligibility, and has to infer that absence means false. If the flag is ever inverted or renamed, that implicit default silently changes meaning for every historical listing.

In setNonCollateral the emission is unconditional even when the stored value does not change, so a call passing the value the market already has rewrites the same slot and emits an event announcing a change that did not happen. Monitoring built on this event will report spurious configuration changes, and the storage write costs gas for nothing.

The event parameter is also named nonCollateral in the contract but disabled in the test interface declaration for the same event, which would surface as two different ABI parameter names for consumers generated from different sources.

The last point and the comparison with Market.setFrozen (which returns early when the value is unchanged) concern sources outside the reviewed artifact — only Auditor.sol.diff is in scope — so they rest on external context rather than the reviewed diff.

**Recommendation**

Emit unconditionally in enableMarket, matching the other three listing events:

```
emit NonCollateralSet(market, nonCollateral);
```

Short-circuit the no-op in setNonCollateral:

```
if (markets[market].nonCollateral == nonCollateral) return;
markets[market].nonCollateral = nonCollateral;
emit NonCollateralSet(market, nonCollateral);
```

Align the event parameter name across the contract and the test interface so a single name appears in every generated ABI.

**Client Comment**

Fixed at 6b1ad9f42fa09b62efc813159305f73baddb0b19.

The no-op guard was left out to keep setNonCollateral consistent with the Auditor's other setters, none of which short-circuit unchanged values.

**/Auditor.sol.diff**

```
391→398 | emit MarketListed(market, decimals);
392→399 | emit PriceFeedSet(market, priceFeed);
```

```

393→400 |         emit AdjustFactorSet(market, adjustFactor);
401 +     if (nonCollateral) emit NonCollateralSet(market, true);

same file

421 +     markets[market].nonCollateral = nonCollateral;
422 +     emit NonCollateralSet(market, nonCollateral);

same file

459 + /// @notice Emitted when a market's collateral eligibility is
    changed by admin.
460 + /// @param market address of the market whose collateral
    eligibility changed.
461 + /// @param nonCollateral true if the market is now ineligible as
    collateral.
462 + event NonCollateralSet(Market indexed market, bool nonCollateral);

```

**CVF-3: acknowledged**Category: **flaw**

checkShortfall reads market data without validating isListed

The diff rewrites the early-return guard in checkShortfall and adds a nonCollateral clause to it:

```

if (accountMarkets[account] & (1 << markets[market].index) == 0 ||
    markets[market].nonCollateral) return;

```

Neither the old nor the new form checks `markets[market].isListed`. For an unlisted market both fields default (`index` to zero, `nonCollateral` to false), so the guard tests bit zero of the account's bitmap — that is, `marketList[0]` — and, if that bit happens to be set, falls through to `accountLiquidity(account, market, amount)`. There `market == marketToSimulate` never matches any entry, so `amount` is silently ignored and the call degrades to a plain solvency check rather than the intended hypothetical-withdrawal check.

The impact is low: `checkShortfall` is a view guard, and its real callers pass a listed market. The missing `isListed` check predates this change, but the diff added a second unvalidated field read (`nonCollateral`) to the same expression, which makes this the natural place to add the guard.

For completeness, the same missing-`isListed` pattern exists in `setPriceFeed` and `calculateSeize`, but the diff does not modify either function — they appear only as unchanged context — so they are outside the scope of this change and would be pre-existing-code observations rather than findings about the diff.

**Recommendation**

In `checkShortfall`, cache the market data once, validate it, and reuse it, which also removes the duplicated mapping lookup on that line:

```
MarketData storage m = markets[market];
if (!m.isListed) revert MarketNotListed();
if (accountMarkets[account] & (1 << m.index) == 0 ||
    m.nonCollateral) return;
```

If pre-existing code is in scope for a later round, `setPriceFeed` and `calculateSeize` share the same gap and could be given the same `isListed` validation; they are left out here because the current diff does not touch them.

### Client Comment

Acknowledged. As the report notes, `checkShortfall` is a view guard and every caller already operates on a listed market, so the check would be redundant.

### /Auditor.sol.diff

```
181→181 |     function checkShortfall(Market market, address account, uint256
amount) public view virtual {
182 | -     // if the account is not 'in' the market, bypass the liquidity
check
183 | -     if ((accountMarkets[account] & (1 << markets[market].index)) ==
0) return;
182 | +     // bypass the liquidity check if the account is not 'in' the
market or it is disabled as collateral
183 | +     if (accountMarkets[account] & (1 << markets[market].index) == 0
|| markets[market].nonCollateral) return;
```

### CVF-4: fixed

Category: **readability**

`checkShortfall` drops the clarifying parentheses around its bitmap-membership test

The diff rewrites the `checkShortfall` guard. It previously read

```
if ((accountMarkets[account] & (1 << markets[market].index)) == 0)
    return;
```

and now reads

```
if (accountMarkets[account] & (1 << markets[market].index) == 0 ||
    markets[market].nonCollateral) return;
```

The rewritten expression is correct: Solidity binds `<<` and `&` more tightly than `==`, so it still parses as `(accountMarkets[account] & (1 << ...)) == 0`. The concern is readability. This precedence is the opposite of C, C++, Java, JavaScript and Python, where `==` binds tighter than `&`, and the same text there would compare the shift result against zero and then AND the boolean. A reader arriving from any of those languages will read

the condition as inverted, and the new outer `|| markets[market].nonCollateral` makes the line long enough that the precedence is not obvious at a glance. This is the guard that decides whether a liquidity check runs at all, so dropping the parentheses trades two characters for a genuine misreading risk.

The change also introduces a local inconsistency: several unchanged sites in the same file already parenthesise the mask before comparing (for example `if ((marketMap & marketMask) == 0)`), while this rewritten site no longer does. Those other sites, and the fact that the inner parentheses around `1 << markets[market].index` are themselves redundant (`<<` binds tighter than `&`), are pre-existing and not modified by this diff, so they are context rather than part of this finding.

### Recommendation

Restore the outer parentheses at the changed site so the intent is unambiguous:

```
if ((accountMarkets[account] & (1 << markets[market].index)) == 0
    || markets[market].nonCollateral) return;
```

If pre-existing code is in scope for a later round, the same parenthesised style could be applied to the loop conditions that currently omit it, or the membership test could be extracted into a small named helper so the intent is spelled out once rather than at every site:

```
function isIn(address account, uint8 index) internal view returns
    (bool) {
    return (accountMarkets[account] & (1 << index)) != 0;
}
```

### Client Comment

Fixed at commit 21deec003a78a50dd3d1073c52cbf9135afbf3c5

### /Auditor.sol.diff

```
183 +   if (accountMarkets[account] & (1 << markets[market].index) == 0
    || markets[market].nonCollateral) return;
```

### CVF-5: acknowledged

Category: **readability**

`checkLiquidation`'s new block reassigns the reused local `value` inside a conditional

`checkLiquidation` uses a single local named `value` for two unrelated quantities. The debt use is unchanged, but the diff wraps the collateral use in a new `if (!marketData.nonCollateral)` block, so `value` is now reassigned conditionally:

```
uint256 value = debt.mulDivUp(m.price, m.baseUnit); // unchanged
base.totalDebt += value;
base.adjustedDebt += value.divWadUp(m.adjustFactor);
```

```

if (!marketData.nonCollateral) {                                // added by
    this diff
    value = collateral.mulDivDown(m.price, m.baseUnit);
    base.totalCollateral += value;
    base.adjustedCollateral += value.mulWadDown(m.adjustFactor);
    if (market == seizeMarket) base.seizeAvailable = value;
}

```

Before the change the two uses were adjacent and unconditional, so the reassignment was easy to follow. Now the second use sits inside a conditional, and the name gives no clue which quantity value holds at the end of the block. No bug results, because value is not read after the block, but giving each quantity its own name would make the new guard read unambiguously.

For context, the reuse of value and the fact that m denotes a MarketVars memory struct in checkLiquidation while it denotes a MarketData storage pointer in enterMarket, exitMarket, accountLiquidity, checkBorrow and handleBadDebt (so the new guard reads marketData.nonCollateral here where the others read m.nonCollateral) both predate this diff — those declarations are unchanged. They are pre-existing observations rather than part of this finding.

### Recommendation

Give the collateral quantity its own name in the new block so no local is reused across the conditional:

```

if (!marketData.nonCollateral) {
    uint256 collateralValue = collateral.mulDivDown(m.price,
    m.baseUnit);
    base.totalCollateral += collateralValue;
    base.adjustedCollateral +=
    collateralValue.mulWadDown(m.adjustFactor);
    if (market == seizeMarket) base.seizeAvailable = collateralValue;
}

```

The first use can likewise be named debtValue for symmetry. Both changes are local and cost no gas.

If pre-existing code is in scope for a later round, consider renaming the two struct variables in checkLiquidation so that m keeps the meaning it has in the rest of the file (for example m for the MarketData pointer and marketVars for the memory struct).

### Client Comment

Acknowledged. value is reassigned and read only within the same conditional block and is not read afterwards. Reusing a short local this way is the existing style across the codebase, and the reuse predates this change.

### /Auditor.sol.diff

```

227 +         if (!marketData.nonCollateral) {
228 +             value = collateral.mulDivDown(m.price, m.baseUnit);
229 +             base.totalCollateral += value;

```



```

230 +         base.adjustedCollateral +=
value.mulWadDown(m.adjustFactor);
231 +         if (market == seizeMarket) base.seizeAvailable = value;

```

**CVF-6: fixed**Category: **efficiency**

checkShortfall's new guard indexes the markets mapping twice

The rewritten guard in checkShortfall indexes markets[market] twice on a single line:

```

if (accountMarkets[account] & (1 << markets[market].index) == 0 ||
    markets[market].nonCollateral) return;

```

Before this change the guard indexed markets[market] only once, for .index; the added || markets[market].nonCollateral clause introduces the second lookup. Each markets[market] hashes the key to derive the struct's base slot before the SLOAD, so the second indexing repeats that hashing for a field that sits in a different slot of the same struct. Every other function in the file caches the entry into a MarketData storage local first, so this line is the outlier.

For context, two related inefficiencies exist in code the diff does not modify and are therefore not introduced by this change: calculateSeize and computeSeize index markets[repayMarket] and markets[seizeMarket] several times without caching, and 10 \*\* decimals is recomputed at run time on every use in accountLiquidity, checkLiquidation, calculateSeize, computeSeize and handleBadDebt, even though decimals is fixed at listing. The handleBadDebt line the diff re-indented into the new conditional carries a pre-existing 10 \*\* m.decimals but already uses a cached m, so it adds no new lookup. These are worth addressing if pre-existing code is in scope, but they are context rather than part of this diff-scoped finding.

**Recommendation**

Cache the struct pointer once, as the rest of the file does, and read both fields from it:

```

MarketData storage m = markets[market];
if (accountMarkets[account] & (1 << m.index) == 0 ||
    m.nonCollateral) return;

```

This computes the struct's base slot a single time and coincides with the caching already suggested for the isListed check on the same line.

If pre-existing code is in scope for a later round, the same caching applies to calculateSeize (hoist markets[repayMarket] and markets[seizeMarket] into locals and pass the seize market's data into computeSeize instead of re-reading it), and a precomputed base unit could be stored in MarketData and written in enableMarket to avoid the repeated EXP:



```
uint64 baseUnit; // = 10 ** decimals, derived cache written at  
listing time
```

A uint64 accommodates any plausible token decimals and fits alongside priceFeed and nonCollateral, keeping the struct at two slots; decimals stays as part of the public getter.

#### Client Comment

Fixed at commit 21deec003a78a50dd3d1073c52cbf9135afbf3c5

#### /Auditor.sol.diff

```
183 +     if (accountMarkets[account] & (1 << markets[market].index) == 0  
|| markets[market].nonCollateral) return;
```

#### CVF-7: acknowledged

Category: **readability**

The solhint suppression on MarketData is unnecessary and detaches the struct's documentation

The change appends bool nonCollateral to MarketData and adds a lint suppression immediately above the declaration:

```
/// @param priceFeed address of the price feed used to query the  
asset's price.  
/// @param nonCollateral true if the market's floating supply is  
ineligible as collateral.  
// solhint-disable-next-line gas-struct-packing  
struct MarketData {  
    uint128 adjustFactor;  
    uint8 decimals;  
    uint8 index;  
    bool isListed;  
    IPriceFeed priceFeed;  
    bool nonCollateral;  
}
```

The suppression is not needed. The members total 40 bytes, so two slots is the minimum regardless of ordering, and the current order already achieves it: adjustFactor, decimals, index and isListed occupy 19 bytes of the first slot, and priceFeed plus nonCollateral occupy 21 bytes of the second. The new field costs no additional slot and no additional SSTORE, so the linter's complaint is about member ordering rather than slot count. Suppressing it hides that fact from the next reader, who will reasonably assume the struct is genuinely under-packed and that the directive records an accepted cost.

The placement is also unfortunate: the directive sits between the natspec block and the declaration it documents, interrupting the @param sequence that describes the struct's members.

### Recommendation

Reorder the members so the linter is satisfied and the suppression can be deleted, grouping the small fields ahead of the address-sized one:

```
struct MarketData {
    uint128 adjustFactor;
    uint8 decimals;
    uint8 index;
    bool isListed;
    bool nonCollateral;
    IPriceFeed priceFeed;
}
```

This still occupies two slots, with 20 bytes in the first and 20 in the second. Changing the declaration order also changes the order of the values returned by the public markets getter, so periphery contracts and tests that destructure it need updating in the same change.

If the ordering is kept as it is, move the suppression above the natspec block so the documentation stays adjacent to the declaration, and extend the comment to record why the rule is suppressed — namely that the struct is already at its minimum of two slots.

### Client Comment

Acknowledged. The rule flags the ordering. Reordering members would change the tuple returned by the public markets getter and break internal and external integrations that decode it by position.

### /Auditor.sol.diff

```
459→483 |    /// @param priceFeed address of the price feed used to query the
asset's price.
484 +    /// @param nonCollateral true if the market's floating supply is
ineligible as collateral.
485 +    // solhint-disable-next-line gas-struct-packing
460→486 |    struct MarketData {
same file
465→491 |        IPriceFeed priceFeed;
492 +        bool nonCollateral;
466→493 |    }
```

**CVF-8:** acknowledged

Category: **procedural**

`enableMarket` changes its selector with no legacy overload kept

Adding the `nonCollateral` parameter changes the function selector of `enableMarket` from `enableMarket(address, address, uint128)` to `enableMarket(address, address, uint128, bool)`. No three-argument overload was retained, so the old selector no longer exists on the upgraded contract. `enableMarket` is restricted to `DEFAULT_ADMIN_ROLE`, so no on-chain contract calls it, and the realistic breakage is off-chain and governance-side rather than protocol-side:

- A timelock or multisig transaction queued before the upgrade that encodes the old three-argument calldata will revert once the implementation is swapped, because the selector is gone.
- Admin tooling, scripts, or subgraph handlers holding a hardcoded ABI for the old signature stop working.

The asymmetry is worth calling out: the public `markets` getter stays backward compatible, because the new field was appended last to `MarketData`, so a legacy five-output decoder still decodes correctly — while the `enableMarket` setter is not compatible.

Some supporting details reference files outside the reviewed artifact (only `Auditor.sol.diff` is in scope) and rest on external context: the deploy script's `hasFunction("setNonCollateral")` capability probe (which handles an auditor that predates the feature, not older tooling addressing an auditor that already has it), the in-repository caller updates in `deploy/Markets.ts` and the test suites, and the `testLegacyMarketsGetterAfterUpgrade` assertion. The selector change itself and the getter's append-only compatibility are verifiable from the diff.

### Recommendation

Either keep a deprecated three-argument overload that forwards with `nonCollateral` set to `false`, preserving the old selector at negligible cost:

```
function enableMarket(Market market, IPriceFeed priceFeed, uint128
    adjustFactor) external {
    enableMarket(market, priceFeed, adjustFactor, false);
}
```

Or, if the break is intentional, state it explicitly in the changeset and release notes as a breaking administrative ABI change, and confirm before the upgrade that no queued governance proposal encodes the old selector.

### Client Comment

Acknowledged. `enableMarket` is admin only, so no external integration uses it. No queued governance proposal encodes the old selector.

### /Auditor.sol.diff

```
371→376 function enableMarket(
372→377     Market market,
373→378     IPriceFeed priceFeed,
374 -     uint128 adjustFactor
379 +     uint128 adjustFactor,
380 +     bool nonCollateral
```

375→381 | `) external onlyRole(DEFAULT_ADMIN_ROLE) {`

### CVF-9: acknowledged

Category: **efficiency**

Order the checkShortfall bypass so the cheaper non-collateral check can short-circuit first

The bypass guard in checkShortfall evaluates the membership test before the non-collateral test:

```
if (accountMarkets[account] & (1 << markets[market].index) == 0 ||
    markets[market].nonCollateral) return;
```

The two `||` operands differ in cost. The membership operand reads two storage slots — `accountMarkets[account]` and `markets[market].index` — and then shifts, ANDs and compares, whereas the `markets[market].nonCollateral` operand reads a single slot. Because `||` short-circuits, placing the cheaper `nonCollateral` operand first lets a non-collateral market — the case this clause was added to handle — return after one field read, skipping the two-slot membership test:

```
if (markets[market].nonCollateral || accountMarkets[account] & (1 <<
    markets[market].index) == 0) return;
```

The reordering is semantically identical, since the guard returns when either condition holds. It is not a strict improvement: when the market is collateral and the account is not in it, the original short-circuits on the membership operand while the reordered form pays one extra `nonCollateral` read. But in the path this guard targets — a non-collateral market — the reordered form is cheaper, and in the common `checkShortfall` path, where the account holds a position in the market it is withdrawing from or transferring, both operands are read either way, so the reorder adds no cost there.

This is a micro-optimization on the same line as CVF-6, which caches the repeated `markets[market]` lookup; the two changes compose.

### Recommendation

Reorder the operands so the single-slot `nonCollateral` check comes first:

```
if (markets[market].nonCollateral || accountMarkets[account] & (1
    << markets[market].index) == 0) return;
```

If the caching suggested in CVF-6 is also applied, the cached pointer lets both reads share the struct's base slot:

```
MarketData storage m = markets[market];
if (m.nonCollateral || accountMarkets[account] & (1 << m.index) ==
    0) return;
```

### Client Comment

Acknowledged. The reorder only saves gas on non-collateral markets, which are the exception. We measured both orders with the gas snapshot suite and keep the current one.

### /Auditor.sol.diff

```
183 +   if (accountMarkets[account] & (1 << markets[market].index) == 0  
    || markets[market].nonCollateral) return;
```

## CVF-10: acknowledged

Category: **efficiency**

enableMarket emits several separate events for a single listing operation

When a market is listed, enableMarket emits up to four separate events — MarketListed, PriceFeedSet, AdjustFactorSet, and, when the flag is set, NonCollateralSet:

```
emit MarketListed(market, decimals);  
emit PriceFeedSet(market, priceFeed);  
emit AdjustFactorSet(market, adjustFactor);  
if (nonCollateral) emit NonCollateralSet(market, true);
```

This has two minor downsides. Each LOG costs gas, so four events cost more than one, though enableMarket is an admin-only, once-per-market operation where that is negligible. More relevant for consumers, the full initial configuration of a market is spread across several log entries that an indexer must correlate to reconstruct the listing.

Both points are limited in force. The granular per-parameter events are a deliberate choice: PriceFeedSet, AdjustFactorSet and NonCollateralSet are the same events the individual setters emit, so a consumer tracking, for example, the current price feed can use one handler regardless of whether the value was set at listing or changed later. And because all four events are emitted in the same transaction for the same indexed market, they can be grouped reliably by transaction hash and log order — the reconstruction is possible, just not contained in a single record. The pattern is also largely pre-existing: the diff only adds the conditional NonCollateralSet emission on the last line; the other three emits are unchanged.

So this is a design trade-off rather than a defect: a single combined listing event is more self-contained, while the current granular events stay uniform with the setters. This also intersects with CVF-2, which notes that the NonCollateralSet emission here is conditional while its three neighbours are unconditional.

## Recommendation

If a self-contained listing record is preferred, add a single combined event carrying the full initial configuration, rather than removing the granular events that the setters also emit:

```
event MarketListed(Market indexed market, uint8 decimals,
    IPriceFeed priceFeed, uint128 adjustFactor, bool nonCollateral);
```

Consumers that only need the initial snapshot can then read one record, while those tracking later changes keep using the per-setter events.

Alternatively, keep the current per-parameter events as-is and document that a listing is reconstructed by grouping the events emitted within the `enableMarket` transaction. Given the negligible gas cost and the reliable same-transaction grouping, either approach is acceptable, so this is best treated as a low-priority observability preference.

### Client Comment

Acknowledged. Extending `MarketListed` would change its signature and break the current event ABI for existing indexers. The granular events are the same ones the setters emit, so consumers use one handler per parameter.

### /Auditor.sol.diff

```
391→398      emit MarketListed(market, decimals);
392→399      emit PriceFeedSet(market, priceFeed);
393→400      emit AdjustFactorSet(market, adjustFactor);
401 +    if (nonCollateral) emit NonCollateralSet(market, true);
```

### CVF-11: acknowledged

Category: **behavior**

A non-collateral deposit does not offset debt owed to the same market

The guard added to the collateral sum applies per market, but the debt sum on the following lines is unguarded, so within a single market an account's deposit is discarded while its borrow from that same market still counts in full:

```
if (!m.nonCollateral) sumCollateral +=
    vars.balance.mulDivDown(vars.price,
        baseUnit).mulWadDown(adjustFactor);

// sum all the debt
sumDebtPlusEffects += vars.borrowBalance.mulDivUp(vars.price,
    baseUnit).divWadUp(adjustFactor);
```

`accountSnapshot` returns the deposit and the debt as two independent quantities and they are never netted. That is correct for ordinary markets, where the deposit counts as collateral and the netting happens implicitly through the health calculation. For a non-collateral market the deposit contributes nothing, so the two figures never meet.

The consequence is that an account holding 1,000 units of supply in a non-collateral market and owing 1,000 units to that same market is treated as having pure debt. Its health is computed against collateral held in other markets, it can be liquidated there,

and it pays the liquidation incentive out of that other collateral, all while holding, in the very same market, a freely withdrawable balance sufficient to repay the debt outright. The economically equivalent position of having neither the deposit nor the debt would leave the account healthy.

This follows from the stated design, in which non-collateral supply contributes nothing to borrowing capacity and is never seizable, so it is not a coding error. It is filed because the outcome is counter-intuitive enough to mislead both users and integrators. A front end that shows a per-market net position, or a user reasoning that their deposit covers their borrow in the same market, will both be wrong about liquidation risk. The remedy available to the account, withdrawing and repaying, is always open to it precisely because the supply is freely withdrawable, but nothing in the protocol prompts it and a liquidation can occur first.

Note that this is not an argument for netting the two figures. Netting would let a non-collateral deposit offset debt, which is a form of borrowing capacity and would contradict the feature's purpose.

### Recommendation

Keep the accounting as it is and address the expectation gap instead.

Document, in the natspec of `setNonCollateral` and in the user-facing description of the feature, that supply in a non-collateral market does not offset debt owed to that same market, and that such a position is liquidated against collateral held elsewhere.

For integrators, the useful signal is already available: `Previewer` reports `isCollateral: false` per market. A front end showing a per-market position should use it to avoid presenting a deposit and a borrow in the same non-collateral market as a net figure, and should warn an account that could clear its own debt by withdrawing and repaying before it becomes liquidatable elsewhere.

If the protocol would rather not carry this hazard at all, the stronger option is to prevent the combination: reject a borrow from a market in which the borrower holds non-collateral supply, or reject such a deposit while a borrow from that market is outstanding. That is a larger behavioural change and should only be considered if the confusion is judged more costly than the restriction.

### Client Comment

Disagree. The protocol never nets supply against debt in any market. A deposit and a borrow in the same market are always evaluated separately, and this change does not alter that. Even in a collateral market, such a position is not netted automatically and the account still has to withdraw and repay. A position in a non-collateral market behaves exactly like one in a market the account has not entered, where liquidation seizes collateral held elsewhere. The outcome is consistent with the rest of the protocol's behaviour.

### /Auditor.sol.diff

```
132 +         if (!m.nonCollateral) sumCollateral +=
vars.balance.mulDivDown(vars.price,
baseUnit).mulWadDown(adjustFactor);
133→133
134→134         // sum all the debt
```



```
135→135 |         sumDebtPlusEffects += vars.borrowBalance.mulDivUp(vars.price,
|         baseUnit).divWadUp(adjustFactor);
```

**CVF-12: fixed**Category: **documentation**

The `setNonCollateral NatSpec` still misdescribes what disabling a live collateral market does

The @dev note on `setNonCollateral` warns about the wrong outcome. As reviewed it read:

```
/// @dev Making a market that accounts use as collateral
|     non-collateral may leave their
|     positions liquidatable.
```

Positions that relied on the market do become unhealthy when the flag is set, since `accountLiquidity` stops counting that balance as collateral. But they do not become liquidatable. The same flag also excludes the balance from `base.seizeAvailable` in `checkLiquidation`, so `maxRepayAmount` returns zero, and `Market.liquidate` rejects that with `ZeroRepay`. At the same time `checkShortfall` returns early for the market, so the owner keeps withdrawing or transferring the balance at full value while insolvent.

The accurate description of the outcome is therefore the opposite of what the note says: affected positions become unhealthy **and unliquidatable**, while their backing stays freely withdrawable. The debt that remains is then liable to be written off by `handleBadDebt`, which is permissionless and is called at the end of every `Market.liquidate`.

The client improved this note in commit `1337db45f6cef578344f6bd0264c7c657f6036a7`, which now reads:

```
/// @dev Disabling a market as collateral may leave positions that
|     rely on it liquidatable
|     or generate bad debt.
```

That is a genuine improvement: the bad-debt consequence is now stated, and it is the more important of the two. The residual inaccuracy is the word "liquidatable", which survives from the original and still points a reader towards the one outcome that cannot occur. An operator reading the current note would reasonably conclude that liquidators will resolve the affected positions, which is precisely what the flag prevents.

This finding replaces a higher-severity finding raised in the draft, at the client's request. The client's position is that the steady-state behaviour is not introduced by this change, and that is correct: `accountLiquidity`, `checkLiquidation` and `handleBadDebt` all iterate only the account's entered-market bitmap, so a deposit in a market the account never entered already fails to count as collateral, is already unseizable, and already does not block bad-debt clearing. What the flag adds is the ability to move an already-entered, already-borrowed-against position into that state by administrative action, which is a



transition no user action can produce. Since that action is admin-only and passes through proposal, vote and timelock, documenting it accurately is the appropriate remedy.

### Recommendation

Replace the word "liquidatable" with what actually happens, so the note warns about the real failure mode:

```
/// @dev Disabling a market as collateral leaves positions that
    rely on it unhealthy and
///    unliquidatable: the balance stops counting as collateral, is
    excluded from the seizable
///    set so liquidation reverts with `ZeroRepay`, and stays
    freely withdrawable by its owner.
///    The remaining debt is then liable to be cleared as bad debt
    against `earningsAccumulator`.
///    Do not enable this flag on a market with outstanding
    collateral-backed debt without first
///    winding that exposure down.
```

The last sentence records, in the code, the operational commitment the client has already made in correspondence, which is the most durable place for it.

Consider also stating in the deployment documentation that `deploy/Markets.ts` proposes `setNonCollateral` automatically whenever the `nonCollateral` value in the market configuration differs from on-chain state, so a one-line configuration change is enough to queue this transition.

### Client Comment

Fixed at 71f7df66a7dd6b716d78f5990b706523ffd9f6a3.

The @dev note on `setNonCollateral` now states that disabling a market as collateral leaves positions that rely on it unhealthy and unliquidatable, that the balance stops counting as collateral and is excluded from the seizable set so liquidation reverts with `ZeroRepay`, that it stays freely withdrawable by its owner, and that the remaining debt is then liable to be cleared as bad debt against `earningsAccumulator`. It closes with an instruction not to enable the flag on a market with outstanding collateral-backed debt without first winding that exposure down.

### /Auditor.sol.diff

```
414 + /// @notice Sets whether a market's floating supply is ineligible
    as collateral.
415 + /// @dev Making a market that accounts use as collateral
    non-collateral may leave their positions liquidatable.
416 + /// @param market address of the market to change collateral
    eligibility for.
417 + /// @param nonCollateral true to make the market ineligible as
    collateral.
418 + function setNonCollateral(Market market, bool nonCollateral)
    external onlyRole(DEFAULT_ADMIN_ROLE) {
```

## 6. Conclusion

The change is small, confined to one contract, and accompanied by a thorough test suite of its own. It implements the design the client described, and each of the four properties claimed for a non-collateral market holds in the code as written. No Critical, Major, or Moderate issue stands in this report.

Two findings raised in the draft were withdrawn after the client's objections were verified against the source, and both deserve recording, because in each case the objection was correct and the draft finding was not.

The draft's only Major finding described an account whose collateral market is switched to non-collateral: its collateral credit disappears, its balance stays freely withdrawable at full value, no liquidator can seize it because the balance is excluded from `seizeAvailable`, and the permissionless `handleBadDebt` then writes the remaining debt off against `earningsAccumulator`. The mechanics of that sequence were confirmed, and indeed the client's own test suite asserts them as intended behaviour. The client's objection was that this is not introduced by the change, and on inspection that is right. `accountLiquidity`, `checkLiquidation`, and `handleBadDebt` each iterate only the account's entered-market bitmap, so a deposit in an un-entered market already fails to count as collateral, is already unseizable, and already does not block bad-debt clearing. The same end state is therefore reachable today without this feature. What the change does add is the ability to move an already-entered, already-borrowed-against position into that state administratively. That action is admin-only and, per the client, passes through a public proposal, a vote, and a timelock, and the client has undertaken not to propose the flag on a market with outstanding collateral-backed debt without first assessing the exposure. On that basis the finding was reduced, at the client's request, to the documentation issue recorded as CVF-12, which the client has since fixed. We note for completeness that one element of the client's reasoning does not hold: `setAdjustFactor` and `setPriceFeed` can strip the value credited to a balance, but neither can leave a balance simultaneously worth its full amount to its owner, freely withdrawable while the account is insolvent, and beyond a liquidator's reach. The new flag can. The governance controls, rather than an equivalence to existing powers, are what make this acceptable.

The second withdrawn finding held that a firewall-disallowed account's non-collateral balance could be neither withdrawn by its owner nor recovered by the protocol. The first half is correct, since `VerifiedAuditor` applies only `Allowed` to `checkShortfall`. The second half was wrong: `VerifiedMarket.lock` burns a locked account's entire floating share balance and sweeps its fixed deposits without consulting the flag, so a recovery path exists. The finding was withdrawn.

The remaining twelve findings are all Minor, and four of them were fixed: the inconsistent

emission of `NonCollateralSet`, two readability and gas observations on the new guard in `checkShortfall`, and the `setNonCollateral` documentation discussed above. The other eight were acknowledged and consciously declined, and in most cases the client's reasoning is sound: a missing `isListed` check is redundant because every caller already holds a listed market; reordering `MarketData` to satisfy a packing linter would change the tuple returned by the public `markets` getter and break integrations that decode it positionally; splitting the listing events would break existing indexers; and the protocol nets a deposit against a borrow in no market, so a non-collateral market behaving the same way is consistent rather than anomalous. One declined item is worth restating for operators rather than developers: `adjustFactor` is still unvalidated, and because it surcharges debt as well as discounting collateral, a market listed with a zero factor would make every account holding that market's bit unable to borrow, withdraw, transfer, be liquidated, or have bad debt cleared. The client considers this pre-existing and out of scope, which it is, but the new `nonCollateral` argument makes a zero factor a more plausible mistake at listing time, since an operator may reasonably assume the factor is irrelevant for a market that grants no collateral power. That finding, CVF-1, raised two points, and its status of acknowledged rather than fixed reflects the more important of them: the commit cited in the client's response broadened the documentation of what `adjustFactor` governs, while the bounds check itself was declined.

Two operational points close this out. `deploy/Markets.ts` proposes `setNonCollateral` automatically whenever the configured value differs from on-chain state, so the transition discussed above is reachable from a one-line configuration change and should be treated with the same care as any other governance action over live positions. And the `enableMarket` signature change removes the old selector without leaving a compatible overload, so any queued proposal or off-chain tool encoding the three-argument form must be updated before the upgrade; the client has confirmed no such proposal is outstanding.



# ABDK

## Consulting

## About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

## Contacts

### Email

[info@abdkconsulting.com](mailto:info@abdkconsulting.com)

### Website

[abdk.consulting](https://abdk.consulting)

### X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

### LinkedIn

[company/abdk-consulting](https://company/abdk-consulting)